



AI RESUME & CAREER ADVISOR PROJECT

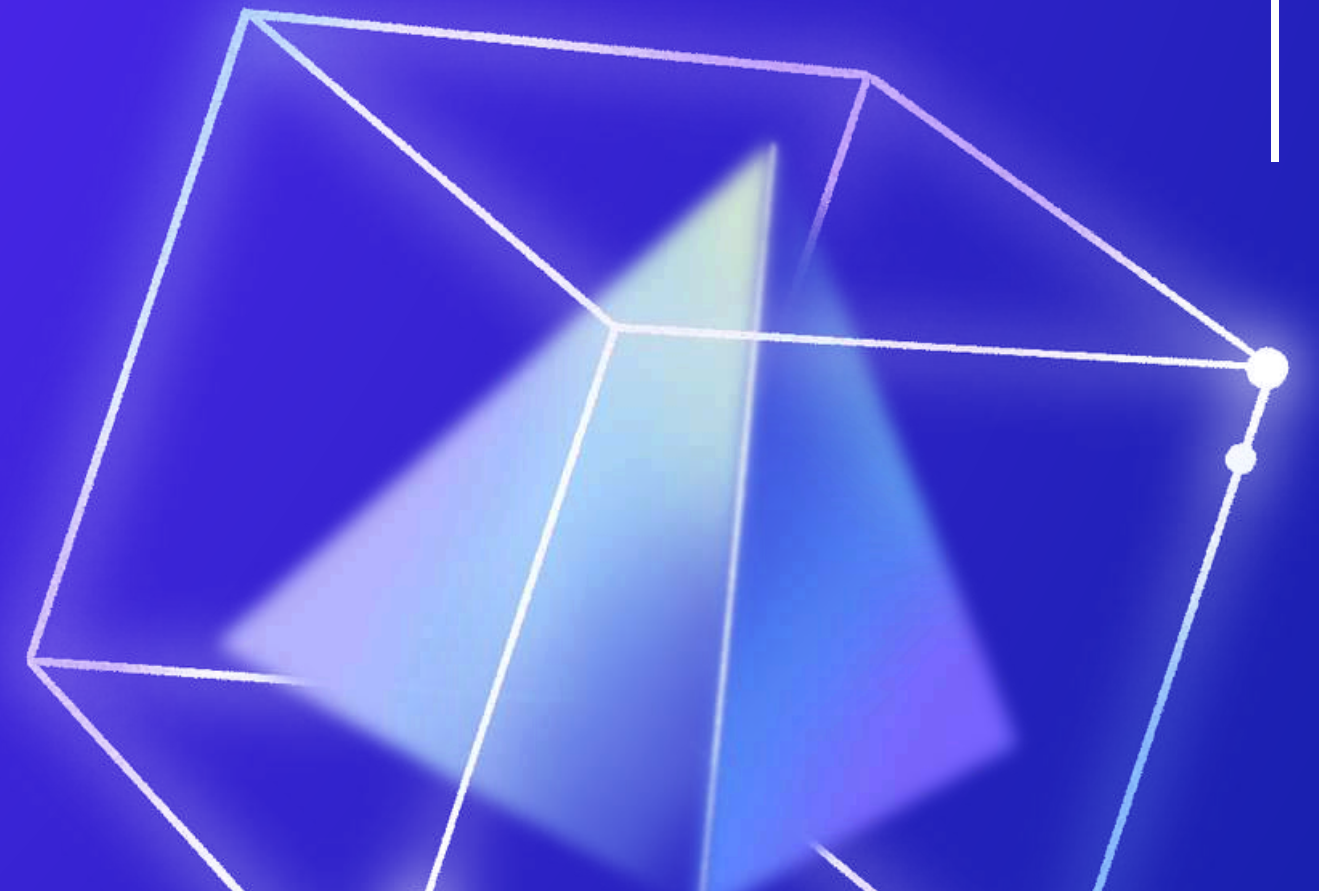
By Saksham Chaurasia





TABLE OF CONTENTS

- The Problem
- Architecture
- Prompt Design
- Methodology
- Evaluation
- Challenges
- Future Enhancements



THE PROBLEM

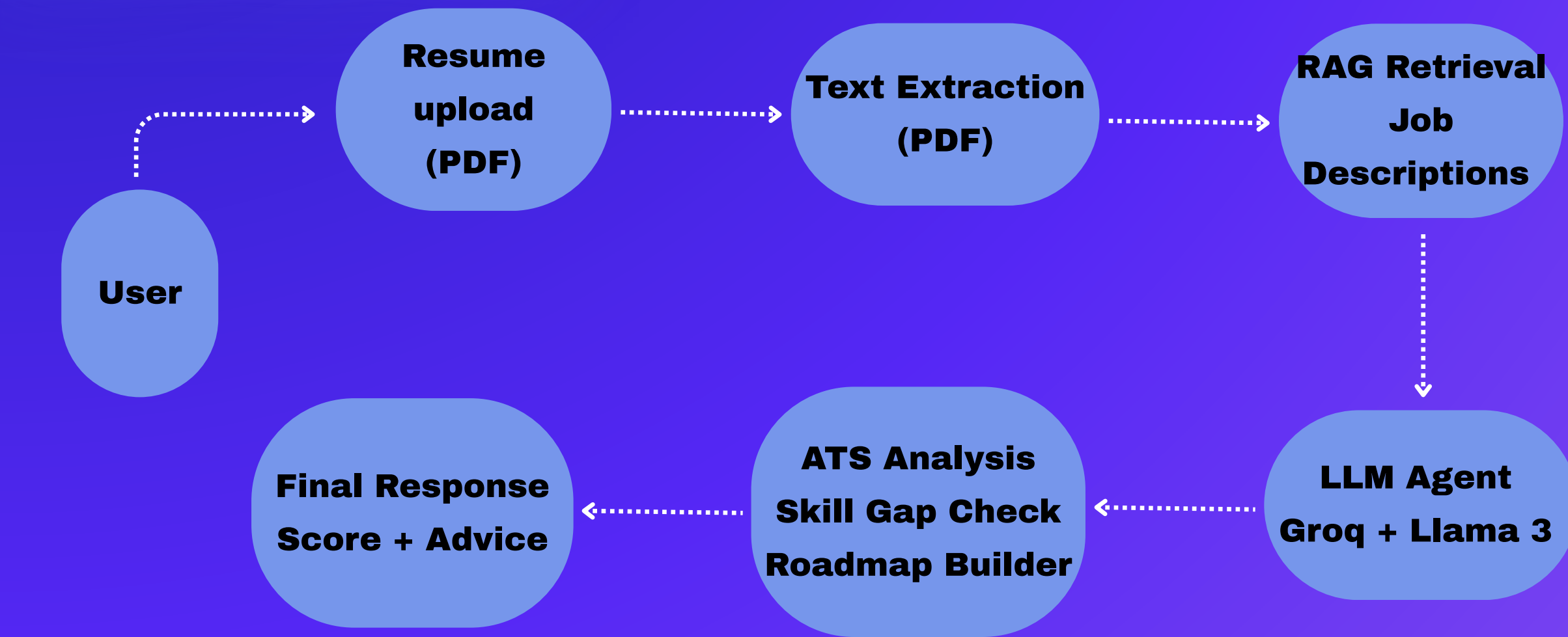
Students often apply for jobs without tailoring their resumes to specific job descriptions.

They do not know:

- Missing skills
- ATS compatibility
- Required technologies
- Learning roadmap
- Interview preparation strategy

The proposed system analyzes a resume, retrieves relevant job requirements from uploaded job descriptions, and generates personalized career guidance.

ARCHITECTURE



PROMPT DESIGN

SYSTEM PROMPT

You are an expert AI Resume & Career Advisor.
Analyze the candidate's resume against the provided job description.
Provide ATS score, strengths, weaknesses, missing skills,
interview preparation guidance, and a personalized learning roadmap.

PROMPT ENGINEERING TECHNIQUES USED :

- **Role Prompting**
- **Model acts as an ATS evaluator and career advisor.**
- **Context Injection**
- **Relevant Job Description retrieved through RAG is provided to the LLM.**
- **Structured Prompting**
- **Output generated in predefined sections.**
- **Task Decomposition**
- **Resume Review**
- **Skill Gap Analysis**
- **Career Guidance**
- **Interview Preparation**

Methodology

Following is the python code we will be using :

```
!pip install groq PyPDF2 python-docx

client =
Groq(api_key="gsk_KhJeaxwyPzoCxb1IweADWGdyb3FYkvKBWckw4Q9
vEsXKI7ga7KmT")

from google.colab import files

uploaded = files.upload()
file_name = list(uploaded.keys())[0]
print("Uploaded file:", file_name)

import PyPDF2
import docx

def extract_text(file_name):
    if file_name.endswith(".pdf"):
        pdf_reader = PyPDF2.PdfReader(file_name)
        text = ""
        for page in pdf_reader.pages:
            text += page.extract_text() or ""
        return text
```

```
    elif file_name.endswith(".docx"):
        doc = docx.Document(file_name)
        return "\n".join([p.text for p in doc.paragraphs])
    else:
        return "Unsupported file type"

def evaluate_resume(text):
    prompt = f"""
You are an expert ATS resume reviewer.
Evaluate the resume and return:
- Score out of 100
- Strengths
- Weaknesses
- Suggestions
- Job readiness level

Resume:
{text}

response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {"role": "system", "content": "You are a strict ATS resume
evaluator."},
        {"role": "user", "content": prompt}
    ]
)
return response.choices[0].message.content
```

```
resume_text = extract_text(file_name)

print("\n===== EXTRACTED TEXT
(Preview) =====\n")
print(resume_text[:1500]) # preview
only

print("\n===== AI EVALUATION =====\n")
result = evaluate_resume(resume_text)
print(result)
```

EVALUATION



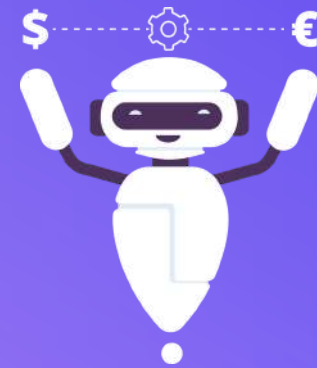
TEST CASE 1

Input: Resume with Java, DBMS, DSA

Retrieved JD: Java Developer

Result:

- ATS Score Generated
- Missing Skills Identified
- Personalized Suggestions Generated



TEST CASE 2

Input: Resume with basic AI/ML skills

Retrieved JD: AI Engineer

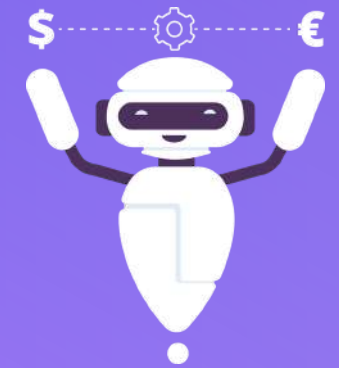
Result:

- Skill Gap Analysis Performed
- Required Technologies Suggested
- Learning Roadmap Generated



OBSERVATIONS

- System successfully analyzed uploaded resumes.
- RAG improved relevance of recommendations.
- Generated responses were context-aware and personalized.



PERFORMANCE METRICS

Metric	Result
Resume Parsing	Successful
Job Retrieval	Successful
ATS Evaluation	Successful
Skill Gap Detection	Successful
Career Guidance	Successful

CHALLENGES



CHALLENGE 1

Resume formats vary significantly.

Solution:

Used PyPDF2 and python-docx for handling multiple file formats.

CHALLENGE 3

Providing job-specific recommendations.

Solution:

Implemented RAG to retrieve relevant job descriptions before evaluation.

CHALLENGE 2

Extracted text sometimes contains formatting issues.

Solution:

Applied text cleaning before sending data to the LLM.

CHALLENGE 4

Maintaining consistent outputs.

Solution:

Used structured prompts and predefined response format.

FUTURE ENHANCEMENTS



01

Vector Database Integration

Replace basic retrieval with :
FAISS, ChromaDB, Pinecone for more accurate retrieval.

02

Multi-Agent Architecture

Replace basic retrieval with – FAISS, ChromaDB, Pinecone for more accurate retrieval.

03

Web-Based Interface

Develop a user-friendly interface using:
Streamlit, Gradio, Flask

04

Real-Time Job Recommendation

Fetch live job descriptions from:
LinkedIn, Naukri, Indeed and perform real-time matching.

05

Resume Optimization

Automatically generate:
ATS-friendly resume versions, Improved project descriptions, Keyword optimization suggestion.
